

K-Means Clustering of UNSW-NB15 Network Traffic: Euclidean vs. Diagonal Mahalanobis Distance

1st Jack Nardi

Dept. of Cybersecurity of Rochester Institute of Technology
Rochester, New York, United States of America
jtn5883@rit.edu

Abstract—Network intrusion detection depends on being able to separate malicious traffic from normal traffic, and clustering offers a way to explore that structure without using labels. In this report, we implemented K-means from scratch using Lloyd’s algorithm with k-means++ initialization and applied it to a class-balanced sample of 3,730 records from the UNSW-NB15 data set, using 39 standardized numeric features. Our implementation was verified against the scikit-learn reference, and the number of clusters was selected as $k = 16$ using silhouette analysis. We then compared standardized Euclidean distance against three diagonal Mahalanobis weightings based on the data set’s feature groups, using V-measure as a primary criterion chosen before any weights were tested. None of the weightings improved on Euclidean distance, which reached a V-measure of 0.316 compared to 0.302 for the best weighted configuration. Analysis of the resulting clusters showed that features we expected to be redundant or meaningless, including traffic volume and TCP sequence-number fields, actually carried useful information for separating the classes.

I. INTRODUCTION

Network intrusion detection systems (NIDS) watch network traffic and try to pick out malicious activity. There are two main types. Signature-based systems compare traffic against known attack patterns, while anomaly-based systems learn what normal traffic looks like and flag anything that doesn’t fit [1]. Signature-based systems can’t catch attacks they haven’t seen before, which is a big reason data-driven approaches have gotten so much attention. To test any of these approaches though, you need a data set that actually looks like real traffic and real attacks.

For a long time, KDDCUP99 and NSL-KDD were the go-to data sets for this. The problem is that both come from simulations run in the late 1990s. They have been criticized for many reasons [1]. UNSW-NB15 was built to fix these issues. It mixes real normal traffic with attack traffic generated by the IXIA PerfectStorm tool, covers nine attack families, and describes each flow with 49 features pulled out using Argus, Bro-IDS, and custom algorithms [1].

Clustering is a way to look at the structure of this data without using the labels at all. If different attack types really sit in different parts of the feature space, K-means [2] should be

able to find groups that line up with them. The catch is that K-means depends completely on how distance is measured, since every point just goes to whichever centroid is closest. The usual approach is to standardize the features and use Euclidean distance, which treats every feature as equally important. That doesn’t seem right for network traffic as a lot of the features measure similar things, like packet counts, byte counts, and load, and some look like they don’t mean much at all. A weighted distance could, in theory, boost the useful features and quiet down the redundant ones.

This report tests that idea on UNSW-NB15. Specifically, we:

- Implemented K-means from scratch using Lloyd’s algorithm with k-means++ initialization [3], and checked it against scikit-learn’s implementation [4].
- Picked the number of clusters without using labels, using elbow and silhouette analysis over $k = 2$ to 20.
- Compared standardized Euclidean distance against three diagonal Mahalanobis weightings based on the data set’s feature groups, using V-measure [5] as the deciding metric, which we chose before testing any weights.

None of the tested weightings beat plain standardized Euclidean distance. Euclidean achieved a V-measure of 0.316, and the best weighted version only achieved a V-measure of 0.302. Looking at the clusters, it turns out that features we expected to be redundant or useless, like traffic volume and the TCP sequence-number fields, actually carry real information about the attack classes.

The rest of this report is laid out as follows. Section II covers the data set and preprocessing. Section III explains the K-means implementation, the Mahalanobis distance, and the evaluation metrics. Section IV checks our implementation against the reference, Section V covers choosing k , and Section VI goes through the weighting experiment. Sections VII and VIII discuss the results and limitations, and Section IX wraps things up.

II. DATA AND PREPROCESSING

The data from this paper comes from the UNSW-NB15 data set [1]. They created the data over two sessions of 15 and 16

hours and labeled it using a ground-truth table from the IXIA tool.

During our training only the training data was used and not the testing CSV. The data contained a large imbalance in the data which would be typical in a realistic scenario but a better distribution of data is required to train the data well. Specifically, the data set contained 56,000 Normal records versus only 130 Worms. For our testing we used a distribution as seen in Table I Additionally the samples were fixed using

TABLE I
CLASS DISTRIBUTION IN THE TRAINING CSV AND THE CLUSTERING SAMPLE

Class	Training CSV	Sample
Analysis	2,000	400
Backdoor	1,746	400
DoS	12,264	400
Exploits	33,393	400
Fuzzers	18,184	400
Generic	40,000	400
Normal	56,000	400
Reconnaissance	10,491	400
Shellcode	1,133	400
Worms	130	130
Total	175,341	3,730

a seed number so the data would be identical between runs but could be changed to test different theories on different data. The sample size was also capped as to reduce the runtime for our from-scratch implementation. The processed data contained 39 features as seen in Table II The attack_cat

TABLE II
NUMERIC FEATURES BY GROUP [1]

Group	Count	Features
Basic	12	dur, spkts, dpkts, sbytes, dbytes, rate*, sttl, dttl, sload, dload, sloss, dloss
Content	8	swin, dwin, stcpb, dtcpb, smean, dmean, trans_depth, response_body_len
Time	7	sinpkt, dinpkt, sjit, djit, tcprtt, synack, ackdat
General purpose	5	is_sm_ips_ports, ct_state_ttl, ct_flw_http_mthd, is_ftp_login, ct_ftp_cmd
Connection	7	ct_srv_src, ct_srv_dst, ct_dst_ltm, ct_src_ltm, ct_src_dport_ltm, ct_dst_sport_ltm, ct_dst_src_ltm
Total	39	

* rate is not in the original feature tables; it is grouped with Basic because it measures traffic volume.

feature is never used for clustering. It is only used for class-balance sampling and evaluation.

III. METHODS

A. K-Means Implementation

We implemented K-means from scratch using Lloyd's algorithm [2]. The main objective was to minimize the inertia

J , which is the sum of squared distances from each point to its assigned centroid. The algorithm works in two steps. First, each point is assigned to its nearest centroid by computing the full distance matrix between all points and centroids and taking the index of the minimum distance for each point. Then each centroid moves to the mean of its assigned points, which "centers" it within its cluster. Each step can only lower J or leave it the same, so the algorithm always converges, but only to a local minimum. Because of this, we run multiple restarts from different starting centroids and keep the result with the lowest inertia.

Initial centroid positions are chosen using standard k-means++ [3], meaning the first centroid is picked at random and each subsequent centroid is picked with a probability proportional to its squared distance to the nearest chosen centroid. If every point already sits on a chosen centroid, the next one is picked uniformly at random instead. If a cluster ends up empty during the update step, its centroid stays at its previous position. The loop stops when the total squared centroid movement drops below 10^{-4} , or after 300 iterations. Labels are reassigned once more after the loop so they match the final centroids.

We used 25 restarts for all primary runs. Early runs with 10 restarts got stuck in noticeably worse local minima, while increasing to 50 restarts did not change the Euclidean result, as discussed in Section IV. A single random generator is seeded once (seed 13) outside the restart loop, so each restart starts from different centroids while the full run stays reproducible. All settings are summarized in Table III.

TABLE III
K-MEANS SETTINGS USED FOR ALL PRIMARY RUNS

Setting	Value
Initialization	k-means++ (standard)
Restarts (n_init)	25
Max iterations	300
Tolerance	10^{-4} (total squared centroid shift)
Empty clusters	Centroid kept at previous position
Random seed	13
Number of clusters	$k = 16$ (Section V)

B. Diagonal Mahalanobis Distance

Next, a simplified Mahalanobis distance was implemented so the different features could be weighted. The simplified version implemented here uses diagonal weight matrix C whose entries c_f are chosen by hand. The idea is that C weights the squared difference between a point and a centroid for each feature before they are summed, so a larger c_f makes feature f matter more to the distance and a smaller c_f makes it matter less. Every weight must be strictly positive, so features are downweighted instead of removed entirely. Having all weights set to 1 gives the same distance as the Euclidean equation.

This weighted distance is also the same as Euclidean distance on data where each feature has been scaled by $\sqrt{c_f}$. This is how the reference implementation is checked for the Mahalanobis runs in Section IV. Since the weights are the

same for every point, they cancel out when finding the best centroid position, so the update step still moves each centroid to the plain mean of its assigned points. The distance is computed in a vectorized way by building every point-to-centroid difference at once, weighting the squared differences, and summing over the features. The k-means++ initialization uses this same weighted distance, so the starting centroids and the assignment step always agree on what "closest" means.

C. Evaluation Metrics

For the evaluation of our models effectiveness a few metrics were used. First was the silhouette score [6], this is a measure of how well each point fits into its own cluster as compared to the next nearest cluster. There are two versions of this metric, the first being common euclidean where both Euclidean and Mahalanobis clusterings are scored in the same Euclidean space and directly compared. This is how k is selected. The other version scores each clustering based on its own distance geometry. The next important metric is homogeneity which looks to see if each cluster only contains one class. Completeness is another metric that measures whether each class stays within one cluster or map to multiple. The next metric is V-measure [5] which measures the harmonic mean of homogeneity and completeness. We selected V-measure as the criterion for our model before any weights were tested. We were looking to improve this metric over the Euclidean baseline. Another important metric is labeling each cluster through majority-vote mapping. This allows each cluster to be labeled by its most common class and this helps when calculating accuracy and macro F1. Additionally due to Majority-vote mapping several clusters can map to one class and some classes can get no clusters. Since the mapping and scoring are both done on the same sample, these scores describe how well the clusters agree with the classes, not how well the model would detect attacks on new traffic. Finally, inertia is not compared between the Euclidean and Mahalanobis runs, since they minimize different objectives.

IV. IMPLEMENTATION VERIFICATION

Our results are verified against the scikit-learn's library using their KMeansReference function with the same k , restarts and seed. This result is then compared to our models results to verify our results. We also used an inertia ratio calculated as $ratio = scratchInertia \div referenceInertia$. A ratio above one means our scratch implementation was worse then scikit's implementation and less than one is better. ARI versus reference helps measure how similar our clustering is compared to scikit's. Essentially our cluster number two might be their cluster number five. Additionally the model contained a couple test functions like `make_test` which allowed us to check the quality of our implementation on a known data set.

These reference checks allowed us to fix a couple of issues that showed up during training. For example our initial run with seed = 42, 10 restarts and $k = 11$ produced an inertia ratio of 1.063 and an increase of inertia from $k = 14$ to 15 which should not happen at a true optimum since adding clusters

should never make the solution worth. This points to restarts landing in poor local minima. Our solution was to increase restarts to 25 and changing the seed to 13. These changes dropped the Mahalanobis ratio to 1.002, a large improvement. Interestingly, our Euclidean ratio to 0.999 meaning that our implementation was slightly better than sklearn's. To try and further this improvements, restarts were increased to 50. This showed no improvement in our model where the sklearn model's inertia improved. This was due to the initialization of k-means. Sklearn uses a greedy implementation of k-means++ that we chose not to implement due to time constraints. Greedy k-means++ is more likely to reach a better solution more often so this explains the difference in results. Our final k was 16 with Euclidean inertia ratio of 1.009. All reference results are shown in Table V.

Additionally there is a loss in ARI at $k=16$ this is due to the fact that at $k = 16$, there are many different clusterings with nearly the same inertia. Two runs can land on different ones that are almost equally good, so ARI drops even though the solution quality is similar. This is why inertia ratio is the main correctness signal rather than ARI.

V. CHOOSING THE NUMBER OF CLUSTERS

During testing we swept k from 2 to 20 with a seed of 13 and 25 restarts. As can be seen in Figure 1 we used an elbow plot, representing inertia changes, and the common Euclidean silhouette. The silhouette was the main guiding factor in choosing a number for k as the elbow graph had a bend to gradual to decide k .

As can be seen in the Elbow graph our inertia flattens out gradually with minimal sharp bends. From $k = 2$ to 9 inertia falls about 65,600 and only about 16,900 from 9 to 16. This suggests there are diminishing returns after nine clusters. Now looking at the silhouette graph we can see a peak at $k=16$. This is also a broad peak with high metrics at 15 and 17 as well.

To arrive at this conclusion we began with exploratory runs with a seed of 42, 10 restarts and a sweep to 15. With our final settings we found the model would consistently peak at 15 indicating that we should expand the boundary of the sweep to 20. This extended sweep clearly showed an interior peak at 16. It is interesting that there are 16 clusters for 10 classes but this indicates that some classes may have several distinct traffic patterns and two clusters could be used to capture a few extreme outliers in our data. Our final results can be seen in Table IV.

VI. MAHALANOBIS WEIGHTING EXPERIMENT

We declared V-measure as our criterion for our experiment before experimenting with any weights. We chose V-measure because it balances homogeneity and completeness and doesn't depend on the majority-vote mapping. The weight allows us to test hypotheses about which features matter most within our data. We came up with a few tests to change data that we thought was less important and we could weight it lower. This came in three stages. Stage one we set *stcpb* and *dtcpb* weights to 0.1, since the TCP sequence numbers appeared

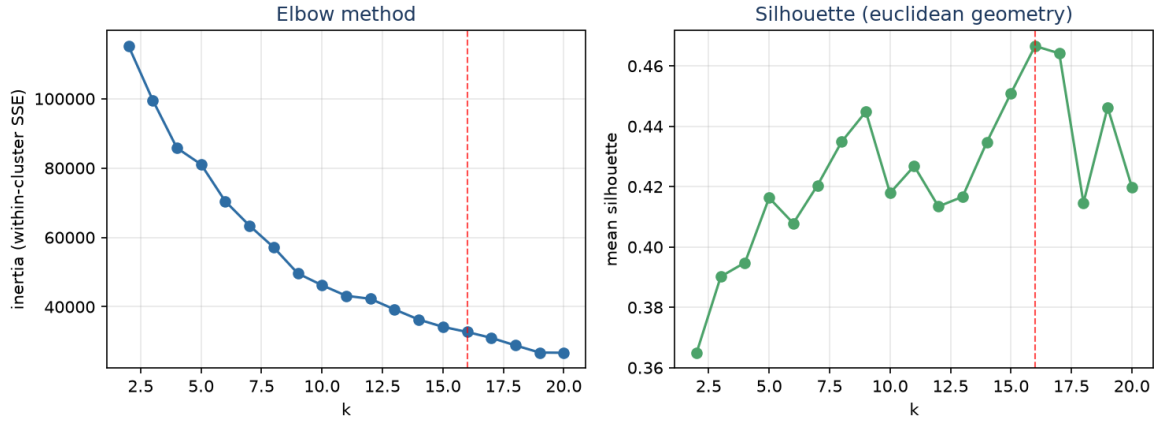


Fig. 1. Elbow (left) and common Euclidean silhouette (right) for the Euclidean sweep over $k = 2$ to 20. The dashed line marks the selected $k = 16$.

TABLE IV
SELECTED VALUES FROM THE EUCLIDEAN k SWEEP

k	Inertia	Silhouette
2	115,181	0.365
9	49,609	0.445
11	43,161	0.427
15	34,209	0.451
16	32,729	0.467
17	31,031	0.464
19	26,796	0.446
20	26,735	0.420

Seed 13, 25 restarts. The selected $k = 16$ is shown in bold. Silhouette is the common Euclidean version.

TABLE V
AGREEMENT WITH THE SCIKIT-LEARN REFERENCE

Run	k	Restarts	Inertia Ratio	ARI
Mahalanobis [†] (seed 42)	11	10	1.063	0.927
Mahalanobis [†]	11	25	1.002	0.977
Euclidean	11	25	0.999	0.896
Euclidean	16	50	1.041	0.855
Euclidean (final)	16	25	1.009	0.781
Mahalanobis Stage 1 (final)	16	25	1.064	0.908

All runs use seed 13 unless noted. [†]Stage 2 weights (Section VI). Ratios above 1 mean the scratch solution has higher inertia than the reference.

to be random values with no behavioral meaning. For stage two we kept the changes from stage one but also set *tcprtt* to 0.1 since the paper defines it as the sum of synack and ackdat, so the handshake time was counted twice. We also set seven other volume features to 0.5 since they measure overlapping data and essentially give that data seven votes in the outcome. For stage three we kept stage 2 but also set *is_ftpt_login* and *is_sm_ips_ports* equal to 0.25, since rare binary flags become extreme values after standardization. Each of these stages builds on the last in an attempt to improve V-measure. The weights must be 0.1 because they must be a strictly positive number. During the trials we used the same values for k , seed and restarts while only the weights changed.

As can be seen in Table VI, none of the weightings improved on Euclidean distance for our primary criterion.

Euclidean clustering reached a V-measure of 0.316, while the best weighted configuration, Stage 1, reached 0.302. The other two candidates scored lower, with Stage 2 reaching 0.283 and Stage 3 reaching 0.296. Since Stage 1 had the highest V-measure of the three candidates, it was selected as our final Mahalanobis configuration.

The rest of the label agreement metrics followed the same pattern as V-measure. Homogeneity, completeness, and the adjusted Rand index were all higher for Euclidean, with the biggest gap in completeness (0.375 versus 0.348). This means the weighted clusters split each class across more clusters than the Euclidean ones did. The majority-vote scores were close but still favored Euclidean, with an accuracy of 0.351 versus 0.347 and a macro F1 of 0.281 versus 0.272. The cluster geometry metrics also favored Euclidean. Both silhouette scores were lower for the Mahalanobis run, and its Davies-Bouldin score was higher (0.888 versus 0.787), meaning its clusters were less compact and less separated. This same direction also showed up in an earlier comparison at $k = 11$, where Euclidean reached a V-measure of 0.304 compared to 0.284 for Stage 2, so the result was not specific to one value of k .

The confusion matrices in Figure 2 show where the differences came from. Generic separated well in both runs, with 353 of its 400 records in Generic-majority clusters for Euclidean and 359 for Stage 1. Normal traffic was the biggest change, with 268 Normal records landing in Normal-majority clusters for Euclidean compared to 206 for Stage 1. Across the candidates this dropped as low as 104 for Stage 2 before recovering to 211 for Stage 3. Worms also moved. Under Euclidean, most Worms records fell into a Fuzzers-majority cluster, while under every weighting about 85 of the 130 Worms records grouped together with Analysis and Exploits records instead. In both runs, Analysis, Backdoor, DoS, Reconnaissance, and Shellcode were largely spread across the same few clusters, showing these classes are hard to separate using flow features alone.

One caveat is that the Mahalanobis run had an inertia ratio

of 1.064 compared to 1.009 for Euclidean, so its solution was a few percent worse than what the reference found, as discussed in Section IV. A better solution could shift the exact numbers slightly, but Euclidean scored higher on V-measure in every comparison we ran, across both values of k and every restart setting tested.

TABLE VI
V-MEASURE BY WEIGHTING STAGE

Stage	Euclidean	Mahalanobis	Difference
Stage 1 (final)	0.316	0.302	-0.014
Stage 2	0.316	0.283	-0.033
Stage 3	0.316	0.296	-0.020
Stage 2, $k = 11$	0.304	0.284	-0.020

Rows above the line use $k = 16$, seed 13, and 25 restarts. The bottom row is the earlier matched comparison at $k = 11$ with the same seed and restarts. Difference is Mahalanobis minus Euclidean.

VII. DISCUSSION

Going into this experiment, our hypothesis was that some features were either redundant or meaningless, and that they were diluting the distance calculation. If that was true, downweighting them should have produced clusters that lined up better with the attack classes. As can be seen in Table VI, the results did not support this. Every weighting we tried lowered the V-measure compared to standardized Euclidean distance. Standardization already gives every feature an equal influence on the distance, and in this data set the features that looked redundant or meaningless turned out to be carrying useful information about the classes.

The clearest example of this is the volume features. Stage 2 halved the weights on the seven features that measure traffic volume, and it produced both the lowest V-measure (0.283) and the lowest completeness (0.314) of any run. Normal traffic was hit the hardest. Under Euclidean distance, 268 of the 400 Normal records landed in Normal-majority clusters, but under Stage 2 that dropped to only 104. This suggests that Normal traffic is separated from the attacks partly by its typical transfer sizes and rates, so shrinking those features blurred it into the attack clusters. The volume features do overlap with each other, but they are not redundant when it comes to separating the classes. Generic, on the other hand, stayed well separated in every run, with between 330 and 361 of its records in Generic-majority clusters, so its separation does not depend much on volume.

The TCP sequence-number fields were another surprise. We expected `stcpb` and `dtcpb` to act as noise since their values are random, but Stage 1, which only downweighted these two features, still lowered the V-measure by 0.014. It also moved most of the Worms records out of a Fuzzers-majority cluster and into a group with Analysis and Exploits records, as can be seen in Figure 2. A likely explanation is that while the values themselves are random, they are 0 for flows that are not TCP connections. Essentially, whether these fields are zero or not acts as a protocol indicator, and downweighting them removed that signal.

Stage 3 recovered some of what Stage 2 lost. Adding the downweighting of the two rare binary flags, `is_ftp_login` and `is_sm_ips_ports`, raised the V-measure from 0.283 to 0.296 and the Normal count from 104 to 211. After standardization, the rare 1 values in these flags become extreme, so they can pull centroids toward a small number of unusual records. Reducing their weight seems to have freed up clusters for real structure. However, Stage 3 also had the worst inertia ratio of any run at 1.087, so its solution was the least reliable and this result should be treated as tentative.

Looking at which classes separated, Generic was consistently clean, Normal separated partially, and Worms formed a recognizable group under the weightings. Analysis, Backdoor, DoS, Reconnaissance, and Shellcode overlapped heavily in every run. This suggests these attacks look very similar at the flow level, with similar connection sizes, timing, and connection counts. Separating them would likely need information that was not used here, such as the categorical protocol and service fields or payload-level features. Additionally, in every run two of the 16 centroids sat on a handful of extreme records far away from everything else. Because K-means uses squared distances, a few very large values in heavy-tailed features like `sbytes` and `sload` can pull an entire centroid toward them. Standardization fixes the scale of these features but not the shape of their distributions, so 2 of our 16 clusters were spent on a few outliers instead of real structure.

It is also interesting that a better solution to the K-means objective did not always mean better agreement with the classes. In our exploratory runs with seed 42 and 10 restarts, Stage 2 appeared to beat Euclidean with a V-measure of 0.318 versus 0.257. That run had landed in a worse local minimum, with an inertia ratio of 1.063, and once more restarts found better solutions the improvement disappeared. K-means minimizes inertia, not disagreement with the attack categories, so a single run can easily be misleading. Overall, the tradeoff of weighting was not worth it for our criterion. The weighted runs had slightly better Generic separation and grouped Worms more consistently, but they lost on V-measure, completeness, silhouette, and Normal separation. Finally, the TTL features were left at their full weight on purpose. Moustafa and Slay note that distinctive TTL values made attacks artificially easy to spot in KDDCUP99 [1], so increasing their weight could have improved our scores for reasons that have nothing to do with real attack behavior.

VIII. LIMITATIONS

There are several limitations to these results. First, all of the evaluation is in-sample. The majority-vote mapping and the scoring were done on the same records, so accuracy and macro F1 describe how well the clusters agree with the classes in our sample, not how well this approach would detect attacks on new traffic. Second, the sample was class-balanced, so it does not reflect the real-world proportions of normal and attack traffic. Third, only the 39 numeric features were used. The categorical fields like protocol, service, and state were

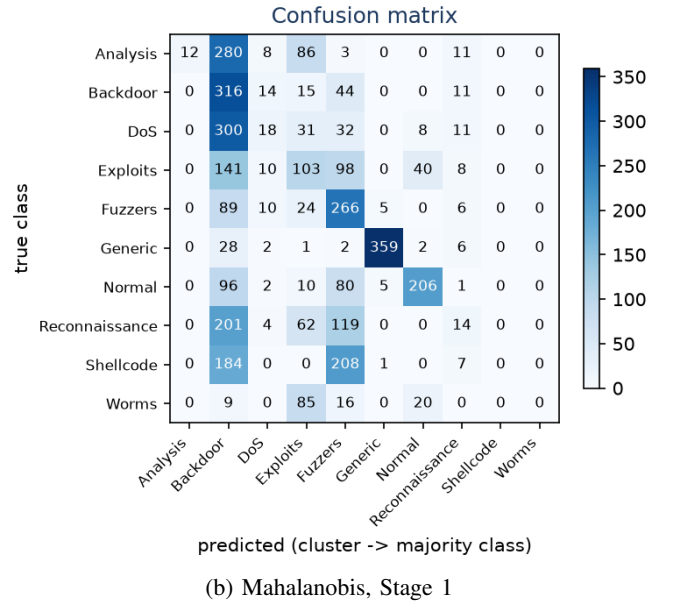
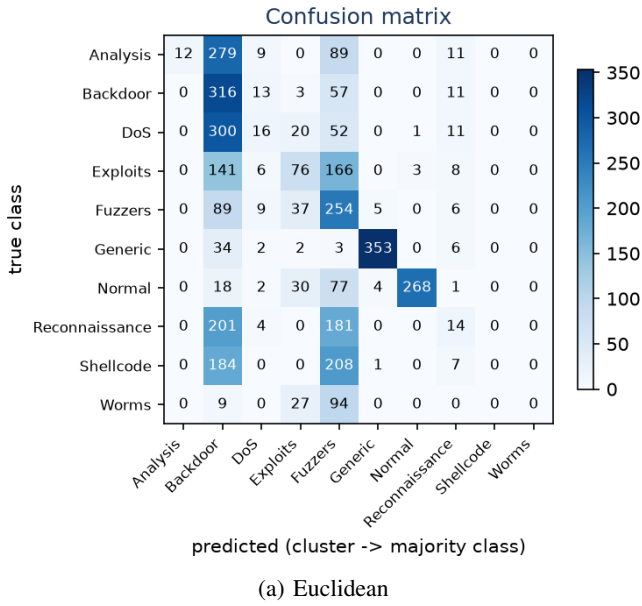


Fig. 2. Confusion matrices after majority-vote mapping at $k = 16$. Rows are true classes; columns are the class each cluster was mapped to.

dropped, and they may have helped separate the classes that overlapped.

The results are also sensitive to local minima. The weighted runs had inertia ratios between 1.04 and 1.09, meaning their solutions were a few percent worse than what the reference found, and a better initialization like greedy k-means++ could shift the exact numbers. Our primary comparison also used a single seed. While Euclidean scored higher in every comparison we ran, testing across more seeds would make that conclusion stronger. Additionally, k was chosen using the Euclidean sweep and then held fixed for the Mahalanobis runs, even though the Mahalanobis sweeps on their own preferred different values of k , so the weighted runs were evaluated at a k chosen for Euclidean distance.

Finally, the weights themselves were chosen by hand based on our hypotheses about the feature groups, and only three candidates were tested. They were not optimized, so a different weighting could still outperform Euclidean distance. The majority-vote mapping also leaves some classes without any cluster, as Shellcode and Worms were never the majority of a cluster in either final run.

IX. CONCLUSION

In this report, we implemented K-means from scratch and used it to cluster a class-balanced sample of the UNSW-NB15 data set. Our implementation closely matched the scikit-learn reference, and the remaining gap at $k = 16$ was traced to the difference between standard and greedy k-means++ initialization. Using the silhouette score, we selected $k = 16$ and compared standardized Euclidean distance against three diagonal Mahalanobis weightings based on the data set’s feature groups. None of the weightings improved on Euclidean distance for our primary criterion, with Euclidean reaching a

V-measure of 0.316 compared to 0.302 for the best weighted configuration.

The main takeaway is that features which seemed redundant or meaningless, like the traffic volume measures and the TCP sequence-number fields, actually carried useful information for separating the classes. Assumptions about which features matter should be tested rather than assumed. For future work, we would implement greedy k-means++ to close the gap with the reference, apply a log transform to the heavy-tailed features to reduce the outlier clusters, include the categorical features, test across multiple seeds, and run a disclosed experiment on the TTL weights.

REFERENCES

- [1] N. Moustafa and J. Slay, “UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set),” in *Proc. Military Communications and Information Systems Conf. (MilCIS)*, Canberra, ACT, Australia, Nov. 2015, pp. 1–6.
- [2] S. P. Lloyd, “Least squares quantization in PCM,” *IEEE Trans. Inf. Theory*, vol. 28, no. 2, pp. 129–137, Mar. 1982.
- [3] D. Arthur and S. Vassilvitskii, “k-means++: The advantages of careful seeding,” in *Proc. 18th Annu. ACM-SIAM Symp. Discrete Algorithms (SODA)*, New Orleans, LA, USA, Jan. 2007, pp. 1027–1035.
- [4] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [5] A. Rosenberg and J. Hirschberg, “V-measure: A conditional entropy-based external cluster evaluation measure,” in *Proc. Joint Conf. Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL)*, Prague, Czech Republic, Jun. 2007, pp. 410–420.
- [6] P. J. Rousseeuw, “Silhouettes: A graphical aid to the interpretation and validation of cluster analysis,” *J. Comput. Appl. Math.*, vol. 20, pp. 53–65, 1987.